Clase 12: Redes - Fundamentos

Introducción: cuando la memoria compartida desaparece

Hasta acá todo lo que coordinamos vivía en la misma máquina. Los procesos se comunicaban por pipes, memoria compartida o señales; los threads compartían directamente el espacio de direcciones. Había un reloj común, la memoria era confiable, y si un proceso moría nos enterábamos.

Nada de eso sobrevive en una red. Dos procesos en máquinas distintas no comparten memoria ni reloj, el canal que los une puede perder, duplicar, demorar o reordenar lo que mandan, y cuando el otro extremo deja de responder no hay forma de distinguir "se cayó" de "está lento" de "se cortó el cable".

Esta clase construye el vocabulario para hablar de eso. Todavía no vamos a escribir un socket —eso es la clase 13— pero sí a entender qué hay abajo: cómo se organizan los protocolos en capas, cómo se identifica a un proceso remoto, y qué garantías da (y cuáles no) cada protocolo de transporte.

> **Nota:** los comandos de esta clase se ejecutan en la terminal. Muchos necesitan herramientas que quizás no tengas instaladas; hay una sección de instalación al final. Varios ejemplos usan servicios públicos de Internet: si estás detrás de un firewall restrictivo, algunos pueden fallar.

Por qué capas

Un mensaje que va de tu navegador a un servidor atraviesa fibra óptica, wifi, routers de varios proveedores y sistemas operativos distintos. Escribir una aplicación que maneje todo eso a la vez sería imposible.

La solución es la misma que en cualquier sistema complejo: dividir en capas, donde cada una resuelve un problema y le ofrece a la de arriba una abstracción más simple.

```
Tu código                    "mandale este texto al servi
    │
Transporte (TCP)             "que llegue completo y en or
    │
Red (IP)                     "que llegue a esta máquina"
    │
Enlace (Ethernet/wifi)       "que llegue a la próxima pla
    │
Físico                       bits, voltajes, ondas
```

Cada capa habla con su par del otro lado como si tuviera un canal directo, aunque en realidad lo que hace es delegar en la capa de abajo. Tu código cree que le manda bytes al servidor; en realidad se los da a TCP, que se los da a IP, y así hasta el cable.

El precio de la abstracción es que las fallas de una capa se manifiestan en otra de forma confusa. Un cable con interferencia (capa física) se ve, desde tu programa, como una conexión lenta. Buena parte del debugging de red consiste en atravesar capas hacia abajo.

Encapsulamiento

Cada capa envuelve los datos de la de arriba con su propio encabezado:

```
[ Ethernet │ IP │ TCP │        datos de tu app        │ ]
 \_________/ \__/ \___/
  14 bytes   20B   20B
```

Los datos de tu aplicación viajan adentro de un segmento TCP, que

viaja adentro de un paquete IP, que viaja adentro de una trama Ethernet. Del otro lado se desenvuelve en orden inverso. Por eso mandar 1 byte de datos cuesta bastante más que 1 byte de tráfico real.

Los dos modelos: OSI y TCP/IP

Vas a encontrar dos modelos de capas y conviene saber cuál es cuál.

OSI (7 capas) es un estándar de los años 80, elaborado por comité. Es exhaustivo y didáctico, pero nunca se implementó tal cual.

TCP/IP (4 capas) es lo que Internet realmente usa. Surgió de implementaciones que funcionaban, y el modelo se escribió después.

OSI	TCP/IP	Ejemplo
7. Aplicación	Aplicación	HTTP, SMTP, DNS
6. Presentación	"	(TLS, codificación)
5. Sesión	"	"
4. Transporte	Transporte	TCP, UDP
3. Red	Internet	IP, ICMP
2. Enlace	Enlace	Ethernet, wifi
1. Física	"	cables, radio

En la práctica se usan mezclados: la gente dice "un problema de capa 3" (vocabulario OSI) hablando de una red TCP/IP. La numeración OSI sobrevivió como jerga aunque el modelo no se implemente.

Un detalle que confunde: OSI separa presentación y sesión, pero en TCP/IP eso es responsabilidad de la aplicación. Cuando en la clase 18 veamos HTTP, vamos a estar programando algo que en OSI abarcaría tres capas.

Direccionamiento: encontrar la máquina

Direcciones IP

Una dirección IPv4 son 32 bits, escritos como cuatro números de 0 a 255:

```
192.168.1.10
```

IPv6 usa 128 bits en hexadecimal, porque las IPv4 se agotaron:

```
2001:0db8:85a3::8a2e:0370:7334
```

Lo vemos en detalle en la clase 16. Por ahora alcanza con saber que conviven.

Algunas direcciones tienen significado especial:

Dirección	Significado
`127.0.0.1`	Loopback: esta misma máquina
`0.0.0.0`	"todas las interfaces" (al hacer bind)
`192.168.x.x`, `10.x.x.x`, `172.16–31.x.x`	Redes privadas (no ruteables en Internet)
`255.255.255.255`	Broadcast en la red local

La distinción entre `127.0.0.1` y `0.0.0.0` va a importar mucho cuando escribamos servidores: si tu servidor escucha en loopback, nadie de afuera lo alcanza; si escucha en `0.0.0.0`, cualquiera que llegue a tu máquina puede conectarse.

Miralo en tu propia máquina:

```
ip addr show           # interfaces y sus direcciones
ip route               # por dónde sale el tráfico
```

Puertos: encontrar el proceso

La IP llega a la máquina, pero ahí puede haber decenas de programas esperando datos. El puerto identifica a cuál.

Un puerto es un número de 16 bits (0 a 65535). El sistema operativo usa el puerto de destino de cada paquete para decidir a qué proceso entregarlo. Eso es **multiplexación**: una sola placa de red sirviendo a muchas aplicaciones.

El rango está dividido por convención:

Rango	Nombre	Uso
0-1023	Bien conocidos	Servicios estándar; en Unix requieren root
1024-49151	Registrados	Aplicaciones con puerto asignado por IANA
49152-65535	Efímeros	Los asigna el SO a los clientes

Algunos que conviene tener memorizados:

Puerto	Servicio
22	SSH
25	SMTP
53	DNS
80	HTTP
443	HTTPS
5432	PostgreSQL
6379	Redis

Que los puertos bajos requieran root no es un capricho: evita que un usuario cualquiera levante un proceso que se haga pasar por el servidor SSH de la máquina.

El socket como par (IP, puerto)

Una conexión TCP no se identifica con un puerto sino con **cuatro** valores:

```
(IP origen, puerto origen, IP destino, puerto destino)
```

Esa cuádrupla es lo que permite que un servidor web atienda a miles de clientes en el puerto 80 simultáneamente: todas las conexiones comparten IP y puerto de destino, pero cada cliente aporta una combinación distinta de origen.

```
Cliente A: (190.1.1.5, 51234) -> (200.16.16.200, 80)
Cliente B: (190.1.1.5, 51235) -> (200.16.16.200, 80)   mi
Cliente C: (181.2.3.4, 51234) -> (200.16.16.200, 80)   ot
```

Las tres son conexiones distintas y el sistema operativo las distingue sin ambigüedad.

Podés ver las conexiones activas de tu máquina:

```
ss -tunap              # todas las conexiones TCP y UDP
ss -tlnp               # solo puertos TCP en escucha
```

Las columnas `Local Address:Port` y `Peer Address:Port` son literalmente los cuatro valores de la cuádrupla.

DNS: de nombres a direcciones

Nadie escribe `200.16.16.200` en el navegador. El DNS traduce

nombres a direcciones, y es un sistema distribuido y jerárquico: ningún servidor conoce todos los nombres, pero cualquiera sabe a quién preguntarle.

```
dig www.um.edu.ar +short      # solo la respuesta
dig www.um.edu.ar             # respuesta completa con TT
host www.um.edu.ar            # alternativa más simple
```

Dos cosas a notar en la salida de `dig`:

- El **TTL** dice cuántos segundos se puede cachear la respuesta. Por eso un cambio de DNS "tarda en propagarse": los caches intermedios siguen sirviendo el valor viejo hasta que expira.
- Un nombre puede resolver a **varias** direcciones. Es la forma más simple de repartir carga entre servidores.

En el código, la resolución ocurre de forma implícita: cuando le pasás `"www.um.edu.ar"` a un socket, alguien tiene que convertirlo en IP antes de mandar el primer paquete. Esa llamada es bloqueante y puede tardar, algo que va a importar cuando lleguemos a asyncio.

TCP y UDP: dos contratos distintos

Ambos protocolos usan puertos y viven sobre IP. La diferencia está en qué garantizan.

TCP: un flujo confiable

TCP ofrece la ilusión de un caño de bytes entre dos procesos:

- **Orientado a conexión**: hay que establecerla antes de mandar datos.
- **Confiable**: lo que se manda llega, o el emisor se entera del fallo. Retransmite lo que se pierde.
- **Ordenado**: los bytes llegan en el orden en que se enviaron.

- **Control de flujo**: si el receptor no da abasto, frena al emisor.
- **Control de congestión**: si la red se satura, baja el ritmo.

El precio es latencia (hay que establecer la conexión y esperar confirmaciones) y estado en ambos extremos.

El handshake de tres vías

Establecer una conexión TCP cuesta un ida y vuelta completo:

```
Cliente                              Servidor
   |                                    |
   |--------- SYN ------------------>|   "quiero conectarm
   |                                    |
   |<-------- SYN-ACK --------------|   "aceptado, yo tam
   |                                    |
   |--------- ACK ------------------>|   "confirmado"
   |                                    |
   |======== conexión lista ========|
```

Por eso una conexión a un servidor lejano "tarda en abrir" aunque después vaya rápido: antes del primer byte útil ya se fueron uno y medio viajes de ida y vuelta.

El cierre es parecido pero de cuatro pasos, porque cada lado cierra su dirección por separado. Un extremo puede terminar de mandar y seguir recibiendo.

TCP es un flujo, no mensajes

Este es el malentendido más frecuente y la fuente de bugs más común al empezar con sockets.

TCP no preserva los límites de los mensajes. Si escribís tres veces:

```
send("HOLA")
send("COMO")
```

```
send("ESTAS")
```

el otro lado puede recibir `"HOLACOMOESTAS"` de una sola vez, o `"HOLACO"` y después `"MOESTAS"`, o cualquier otra partición. TCP garantiza que los bytes llegan todos y en orden, no que lleguen agrupados como los mandaste.

Si tu aplicación necesita mensajes, tenés que delimitarlos vos: con un separador (`\n`), con un prefijo de longitud, o con un formato autodelimitado. Lo vamos a implementar en la clase 13.

UDP: datagramas sueltos

UDP es casi un IP crudo con puertos:

- **Sin conexión**: se manda y listo, no hay handshake.
- **No confiable**: si se pierde, se perdió. Nadie retransmite.
- **Sin orden garantizado**: el paquete 2 puede llegar antes que el 1.
- **Preserva límites**: un `send` es un `recv`. Lo que mandaste como un datagrama llega como un datagrama (o no llega).

Esto último es una ventaja real sobre TCP en ciertos casos: si tus datos ya son mensajes discretos, UDP no te obliga a delimitarlos.

Cuándo cada uno

	TCP	UDP
Garantía de entrega	Sí	No
Orden	Sí	No
Límites de mensaje	No (flujo)	Sí (datagrama)
Handshake	Sí (3 vías)	No
Overhead de encabezado	20 bytes	8 bytes

Estado por conexión	Sí	No

TCP cuando la integridad importa más que la latencia: web, correo, bases de datos, transferencia de archivos, SSH.

UDP cuando llegar tarde es peor que no llegar: streaming de video y voz, juegos en tiempo real, DNS (una consulta y una respuesta chicas), métricas y telemetría.

El caso de DNS es ilustrativo: la consulta entra en un datagrama y si se pierde, el cliente reintenta. Montar una conexión TCP para eso sería más caro que el reintento ocasional. Pero cuando la respuesta es grande, DNS usa TCP.

Un matiz: "UDP es más rápido que TCP" es una simplificación. UDP tiene menos overhead, pero si tu aplicación necesita confiabilidad y la implementás vos arriba de UDP, es probable que termines con algo más lento y con más bugs que TCP. HTTP/3 sí lo hace, sobre UDP, pero eso son décadas de trabajo de gente muy especializada.

Ver la red en funcionamiento

Antes de programar sockets conviene poder observar lo que pasa. Estas herramientas van a acompañarnos todo el bloque.

ping: ¿llega algo?

```
ping -c 4 8.8.8.8
```

Usa ICMP, no TCP ni UDP. Que responda significa que hay ruta hasta el host. Que no responda **no** significa que esté caído: muchos servidores filtran ICMP.

traceroute: ¿por dónde va?

```
```

```
traceroute www.um.edu.ar
```

Muestra los routers intermedios. Los `* * *` son saltos que no responden, algo normal.

ss: ¿qué está escuchando acá?

```
ss -tlnp              # puertos TCP en escucha, con el pr
```

Es la primera herramienta a usar cuando un servidor "no anda": si tu proceso no aparece acá, no está escuchando donde creés.

netcat: hablar con cualquier puerto

`nc` abre conexiones TCP o UDP crudas. Es el equivalente de red de `cat`, y sirve tanto de cliente como de servidor.

Como cliente, hablando HTTP a mano:

```
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection
```

Como servidor, escuchando en un puerto:

```
nc -l 8080            # queda esperando una conexión
```

Y desde otra terminal:

```
echo "hola" | nc localhost 8080
```

Eso es, conceptualmente, el servidor que vamos a escribir en Python la clase que viene. Vale la pena hacerlo ahora con `nc` para tener la imagen mental antes de pelear con la API.

> **Ojo con los finales de línea**: los protocolos de texto de Internet usan `\r\n`, no `\n`. Un servidor HTTP estricto puede rechazar una petición

> que use solo `\n`. Por eso el `printf` de arriba los pone explícitos.

tcpdump: ver los paquetes

```
sudo tcpdump -i lo -n port 8080
```

Captura el tráfico en la interfaz loopback (`-i lo`) del puerto 8080. Si corrés esto mientras hacés el ejemplo de `nc`, vas a ver el handshake de tres vías, los datos, y el cierre.

Requiere privilegios porque leer todo el tráfico de una interfaz es, justamente, lo que hace un sniffer.

Instalación de herramientas

En Debian/Ubuntu:

```
sudo apt install iproute2 dnsutils netcat-openbsd tracero
```

`iproute2` trae `ip` y `ss`; `dnsutils` trae `dig` y `host`.

Hay varias versiones de netcat con banderas incompatibles. `netcat-openbsd` es la que corresponde a los ejemplos de acá. Verificá con `nc -h` si algo no funciona como se espera.

Conceptos clave

1. **Las capas aíslan problemas**: cada una resuelve lo suyo y le simplifica la vida a la de arriba.
2. **IP identifica la máquina, el puerto identifica el proceso**: hacen falta los dos.
3. **Una conexión es una cuádrupla**: (IP y puerto origen, IP y puerto

destino). Por eso un puerto sirve para miles de conexiones.
4. **TCP es un flujo de bytes, no de mensajes**: si necesitás mensajes, delimitalos vos.
5. **UDP preserva límites pero no garantiza nada**: un datagrama llega entero o no llega.
6. **`127.0.0.1` no es lo mismo que `0.0.0.0`**: determina quién puede alcanzar tu servidor.
7. **En red, lo que en local era imposible pasa siempre**: pérdidas, reordenamientos, y la imposibilidad de distinguir un proceso lento de uno caído.

Preparación para la próxima clase

En la **clase 13 (Sockets TCP)** vamos a escribir en Python lo que acá hicimos con `nc`: crear un socket, hacer `bind`, `listen`, `accept`, y mantener una conversación cliente-servidor. También vamos a chocarnos de frente con el problema de la delimitación de mensajes.

Para llegar preparado:

- Tené las herramientas instaladas y probadas.
- Corré el ejemplo de `nc` cliente/servidor hasta que te salga sin pensarlo.
- Hacé la autoevaluación: los conceptos de esta clase se usan como vocabulario en todo el bloque.

Referencias

- RFC 793 - TCP (el original; ver también RFC 9293, que lo actualiza)
- RFC 768 - UDP (tres páginas: vale la pena leerlo entero)
- Registro de puertos de IANA
- Kurose & Ross, *Computer Networking: A Top-Down Approach* - capítulos 1 a 3

- Stevens, *UNIX Network Programming, Vol. 1* - la referencia clásica de sockets

---